CS 532: Homework Assignment 3

Due: April 8th 6:00PM

Enrique Dunn
Department of Computer Science Stevens
Institute of Technology
edunn@stevens.edu

Collaboration Policy. Homeworks will be done individually or in already formed pairs. Use of the Internet is allowed, but should not include searching for previous solutions or answers to the specific questions of the assignment. I will assume that, as participants in a graduate course, you will be taking the responsibility of making sure that you personally understand the solution to any work arising from collaboration.

Late Policy. 3% per partial 24-hour delay

Submission Format. Electronic submission on Canvas is mandatory. Submit in a zip file, a single pdf file containing:

- the source code,
- brief of explanations of what was done,
- images and screenshots of the model.
- final ply model.

Problem 1. Download the *dancer* dataset from the course web page. The zip file contains images, silhouettes and projection matrices for a single time instant of a dynamic scene. A simple ply file containing 8 points is also included. (Finally, it contains a Matlab script that may be helpful, but is not necessary for completing this assignment.) The goal is to obtain a voxel-based reconstruction of the scene using the provided silhouettes as inputs.

Step 1. Define a voxel grid with x ranging from -2.5 m to 2.5 m, y from -3 to 3 m and z from 0 to 2.5 m. Set the size of each voxel so that the total number of voxels fits comfortably in the memory of your computer. You can start at lower resolution initially to accelerate development. You may also chose to make the initial volume tighter, but this is strictly optional. In this part, the goal is to estimate which voxels are occupied and which are free space.

Step 2. Determine the voxels forming the visual hull. Start with all voxels labeled as EMPTY. Project the 3D center of each voxel to all images and label as OCCUPIED only those voxels whose projection resides inside ALL the silhouettes. Report the total number and what relative portion of all voxels are in the visual hull.

Step 3. Identify voxels that are in the reconstructed surface. Surface voxels are OCCUPIED voxels having at least one neighboring voxel labeled as EMPTY. Report the total number and what relative portion of all voxels are surface voxels.

Step 4. Determine the set of 3D points to include in the output. From the surface voxels attained in Step 3, add to the output 3D model the 3D point of CENTER OF EACH FACE that neighbors with an EMPTY VOXEL. Report the number 3D point included in the model and compare with the number voxels in the surface. Are they the same? Discuss why this is the case.

Step 5. Determine a false-color RGB for each output 3D point. To color each point first determine the range bounding box (i.e. the min and max values in each spatial dimension, i.e. X, Y, and Z) of "reconstructed" points. This will give you X_min, X_max, Y_min, Y_max, Z_min, Z_max. For each single 3D point assign for the RED color channel a value proportional to that point X coordinate by RED=255*(X-X_min)/(X_max-X_min). Color the GREEN and BLUE channel similarly using the Y and Z dimensions

Step 6. Write a PLY file. The output should be in ASCII ply format. Use the included ply file as a sample for how to create such as file. The number after element vertex is the total number of vertices and has to be correct for the model to be displayed correctly. Each vertex is represented by three floating point numbers for the coordinates and three unsigned characters for the RGB colors. Models can be visualized using Meshlab, which is available at http://meshlab.sourceforge.net/ and works on Windows, MacOSX and Linux.

Step 7. Determine a true-color RGB for each output 3D point. Color the model by assigning the per-channel median RGB values of the pixels to which a surface voxel projects into. When computing the color for each voxel take visibility into account and only compute the median RGB values from the set of cameras/images that have a non-occluded view of the voxel***. A potential approach*** is as follows: To test visibility determine a vector for each surface vertex that points outward from the occupied volume to the cameras. If the voxels intersected along these vectors are ALL EMPTY then the camera can "see" the surface voxels. You are welcome/encouraged to try your own method.

Step 8. Write a PLY file.

Acknowledgement. The input images, silhouettes and calibration parameters have been generated and made available by the